Koneru Lakshmaiah Education Foundation
(Category -1, Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
❖ Approved by AICTE ❖ ISO 21001:2018 Certified
Campus: Green Fields, Vaddeswaram - 522 302, Guntur District, Andhra Pradesh, INDIA.
Phone No. +91 8645 - 350 200; www.klef.ac.in; www.klef.edu.in; www.kluniversity.in
Admin Off: 29-36-38, Museum Road, Governorpet, Vijayawada - 520 002. Ph: +91 - 866 - 3500122, 2576129

PARKING LOT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

by

M.N.V. PRASAMSA REDDY - 2601050056

under the supervision of

Dr. Raju Anitha

Professor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation (KLEF)

Vaddeswaram, Guntur District, Andhra Pradesh, India

October 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our respected Principal, Head of the Department, and project guide for providing me with the opportunity and guidance to complete the project titled **"Parking Lot Management System."**

I am thankful to my project guide for their valuable suggestions, continuous support, and encouragement throughout the development of this project. Their guidance helped me understand the concepts of object-oriented programming, greedy slot allocation, and multithreading in Python.

I would also like to thank the faculty members of the department for their support and for providing the necessary resources to carry out this project successfully.

Finally, I express my heartfelt thanks to my parents and friends for their encouragement and support throughout the completion of this project.

M.PrasamsaReddy
M.Tech – Computer Science and Engineering

DECLARATION

I hereby declare that the project entitled “Parking Lot Management System” is prepared and submitted by me as part of the academic requirements for the Master of Technology (M.Tech) programme in Computer Science and Engineering.

This project has been developed using Python programming to manage vehicle entry, parking slot allocation, vehicle search, removal and billing.

I further declare that the information presented in this report is true to the best of my knowledge.

Place: Vaddeswaram

Date: 12-10-2026

TABLE OF CONTENTS

S. No.	Chapter / Section	Page No.
1	Cover Page	1
2	Table of Contents	2
3	Abstract And Introduction	3
4	Real-World Problem And Objectives	4
5	Computational Thinking	5
6	Problem Decomposition And Abstraction	6
7	Algorithm And Data-Structure Selection	7
8	System Architecture And Python Implementation	8
9	Oop And Design Patterns	9
10	Concurrency And Thread Synchronization	10
11	Testing And Quality Engineering	11
12	Ci/Cd And Containerisation	12
13	Ai-Assisted Development And Code Review	13
14	Performance Evaluation And Live Demonstration	14
15	Conclusion, Future Scope And References	15

Abstract

The Parking Lot Management System is a Python-based application developed to manage vehicle entry, parking-slot allocation, vehicle removal and billing in a multi-level parking facility. Manual parking management can cause delays, inefficient slot utilization, incorrect billing and difficulty in tracking parked vehicles. This project addresses these issues through an automated software solution. The system organizes parking spaces into multiple floors, with each floor containing a collection of parking slots. When a vehicle enters the parking facility, the system searches for an available slot and assigns the first available slot according to a greedy allocation strategy. Different vehicle categories, such as cars, bikes and trucks, are represented using object-oriented programming.

The Factory Design Pattern is used to create vehicle objects according to their types. A dictionary maintains vehicle registration numbers and their corresponding parking locations, enabling efficient vehicle lookup. The billing module calculates the parking fee based on the vehicle category and the parking duration provided during vehicle removal. Multithreading is demonstrated by allowing multiple vehicle-parking operations to be initiated concurrently. A thread lock protects critical parking operations and helps prevent multiple threads from modifying shared parking data simultaneously.

The application demonstrates important software engineering concepts, including computational thinking, data structures, algorithms, modular programming, object-oriented design, concurrency, testing and performance evaluation.

Keywords — Parking Lot Management System, Python, Greedy Algorithm, Slot Allocation, Object-Oriented Programming (OOP), Factory Design Pattern, Data Structures, Multithreading, Thread Synchronization, Parking Fee Calculation, Vehicle Management, Software Testing.

1.Introduction

Parking facilities in shopping centres, colleges, hospitals, offices and commercial buildings often manage many vehicles simultaneously. In a manual system, staff may need to identify empty spaces, record registration numbers, track vehicle locations and calculate parking fees. These activities can become difficult when the number of vehicles increases. A software-based parking system can simplify the process by maintaining vehicle records and allocating slots automatically.

The proposed system uses Python because it supports object-oriented programming, built-in data structures, exception handling and multithreading. Its modular design separates vehicle management, slot management, billing and parking operations into different components.

1.1 Scope of the Project

The scope of the current implementation includes:

- Managing parking spaces across multiple floors.
- Supporting cars, bikes and trucks.
- Automatically allocating available parking slots.
- Preventing duplicate registration numbers from being parked simultaneously.
- Displaying parking-slot status.
- Searching for a parked vehicle.
- Removing vehicles and calculating parking fees.
- Demonstrating concurrent parking operations using threads.

2 Real-World Problem And Objectives

Real-World Problem

A multi-level parking facility must manage a limited number of parking spaces while vehicles continuously enter and leave. Manual allocation can lead to inefficient use of space, incorrect vehicle records, longer waiting times and billing mistakes. When multiple entry operations happen simultaneously, the system must also protect shared data from conflicting modifications. Therefore, a structured parking management application is needed to automate slot assignment, vehicle tracking, fee calculation and synchronization of concurrent operations.

2.1 Problem Statement

To design and implement a Python-based Parking Lot Management System that manages vehicle entry and exit, allocates available slots using a greedy strategy, maintains vehicle records, calculates parking charges and supports concurrent parking operations with appropriate thread synchronization.

2.2 Objectives

1. To organize parking slots across multiple floors.
2. To allocate the first available slot automatically.
3. To represent different vehicle types using inheritance.
4. To use a Factory Design Pattern for vehicle-object creation.
5. To maintain vehicle registration numbers and parking locations.
6. To calculate parking fees based on vehicle type and duration.

2.3 Functional Requirements

Requirement	Description
Vehicle entry	Accept vehicle type and registration number
Slot allocation	Assign an available parking slot
Vehicle search	Search using registration number
Vehicle exit	Remove the selected parked vehicle
Billing	Calculate the parking fee
Slot display	Show available and occupied slots
Thread demonstration	Start concurrent parking operations

3 Computational Thinking

Computational thinking is the process of solving problems systematically by breaking them into smaller tasks and designing logical solutions that can be implemented by a computer. The Parking Lot Management System applies four important computational-thinking concepts.

3.1 Decomposition

The complete parking-management problem is divided into smaller modules:

- Vehicle management
- Parking-slot management
- Floor management
- Parking-lot management
- Billing

3.2 Pattern Recognition

Similar operations occur throughout the parking workflow. Every vehicle requires a registration number, every parking slot has an availability status, and every vehicle exit requires record removal. The system identifies these common patterns and uses reusable classes and methods to perform them consistently.

3.3 Abstraction

Abstraction means representing only the information needed for a particular task. For example, a parking slot needs to store its slot number and the vehicle occupying it. The application does not need to model every physical detail of a real parking space. Similarly, the billing module requires a vehicle category and parking duration to calculate the fee.

3.4 Algorithmic Thinking

The system follows a defined sequence:

1. Accept vehicle details.
2. Validate the vehicle type and registration number.
3. Check whether the registration number is already parked.
4. Search floors and slots in order.
5. Select the first available slot.
6. Update the slot and vehicle-location records.
7. Display the parking confirmation.

During vehicle exit, the system locates the vehicle, removes it from its slot, updates the records and calculates the fee.

4 Problem Decomposition And Abstraction

4.1 System Modules

Module	Responsibility
main.py	Displays the menu and accepts user input
vehicle.py	Defines the base Vehicle class and vehicle subclasses
vehicle_factory.py	Creates vehicle objects based on vehicle type
parking_slot.py	Represents a parking slot and its occupancy
parking_floor.py	Maintains slots and searches for an available slot
parking_lot.py	Coordinates parking, removal, search and synchronization
billing.py	Calculates parking charges

4.2 Vehicle Abstraction

The system models vehicles through a common base class. Car, Bike and Truck classes represent specific vehicle categories. Common properties and operations can be defined in the parent class and reused by subclasses.

4.3 Parking-Slot Abstraction

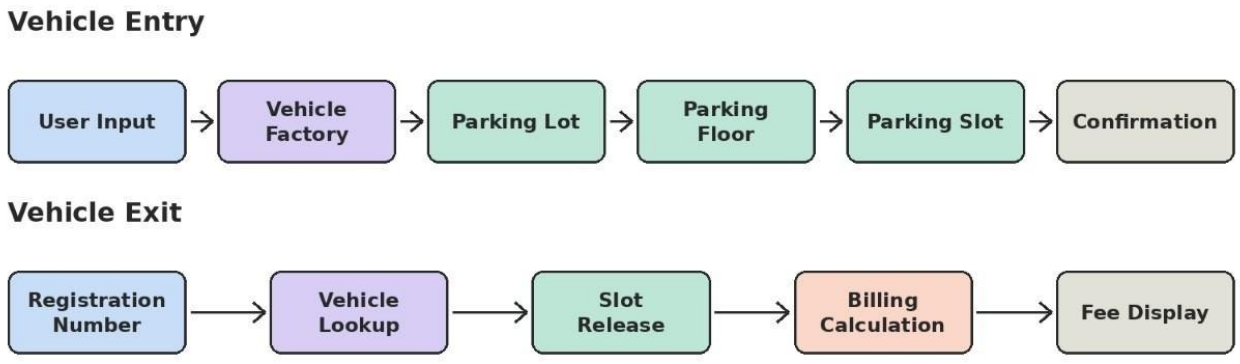
Each parking slot has a unique slot number and an occupancy state.

4.3.1 If the slot does not contain a vehicle, it is available.

4.3.2 If the slot contains a vehicle, it is occupied.

4.3.3 When a vehicle leaves, the slot becomes available again.

4.4 Data Flow



5 Algorithm And Data-Structure Selection

5.1 Algorithm: Vehicle Parking

1. Start.
2. Receive the vehicle type and registration number.
3. Check whether the registration number already exists in the parked-vehicle records.
4. If it exists, display an error and stop the operation.
5. Create the vehicle object using the factory.
6. Traverse the floors in order.
7. Search for the first available slot on each floor.
8. If a slot is available, assign the vehicle to it.
9. Store the floor and slot location in the vehicle dictionary.
10. Display the successful parking message.
11. If no slot is available, display a parking-full message.
12. Stop.

Pseudocode

```
PARK_VEHICLE(vehicle_type, registration_number): IF registration_number is already parked:

    DISPLAY duplicate vehicle error RETURN

    vehicle = CREATE_VEHICLE(vehicle_type) FOR each floor in parking floors:

    slot = FIND_AVAILABLE_SLOT(floor)


    IF slot exists:

    PARK vehicle in slot SAVE vehicle location

    DISPLAY parking confirmation RETURN


    DISPLAY parking full
```

5.2 Complexity Analysis

Let (F) represent the number of floors and (S) the number of slots per floor.

1. **Slot allocation:** Worst-case (O(FS)), because the system may examine every slot.
2. **Vehicle lookup:** Average-case (O(1)) dictionary lookup, assuming normal Python dictionary performance.
3. **Vehicle removal:** Average-case (O(1)) for locating the stored vehicle location, excluding any additional work.
4. **Slot display:** (O(FS)), because the application displays all slots.

6 System Architecture And Python Implementation

6.1 System Architecture

The application uses a modular architecture in which the user interface communicates with the parking-lot controller. The controller interacts with vehicle creation, floor and slot management, and billing components. **Architecture:**

```
User

| v

main.py

| v

parking_lot.py

/ |     \

     v            v      v Vehicle    Floor  Billing Factory            |

|                v

   v           Parking Slots Vehicle
```

The architecture separates the menu, parking logic and supporting components. This makes individual classes easier to modify without rewriting the entire application.

6.2 Development Environment

- Programming language: Python
- Code editor: Visual Studio Code
- Terminal: VS Code integrated terminal or Windows Command Prompt
- Version control: Git and GitHub, if used in the submission workflow
- External database: Not required in the current console implementation

6.3 Implementation Approach

The program is divided into separate Python files. The main.py file manages menu interaction and calls the methods of the parking-lot object. The vehicle module defines the supported vehicle categories. The factory creates objects based on the selected category. The parking-slot module manages occupancy, while the floor module searches its slots. The parking-lot module coordinates the main operations and maintains a dictionary of parked vehicles. The billing module determines the applicable hourly rate based on vehicle category.

6.4 Execution Procedure

1. Open the project folder in Visual Studio Code.
2. Open the integrated terminal.

3. Navigate to the project folder.

4. Execute the application using:

   python main.py

5. Select the required menu option.

6. Enter the requested vehicle details.

7. Observe the confirmation, slot status or billing output.

6.5 Output

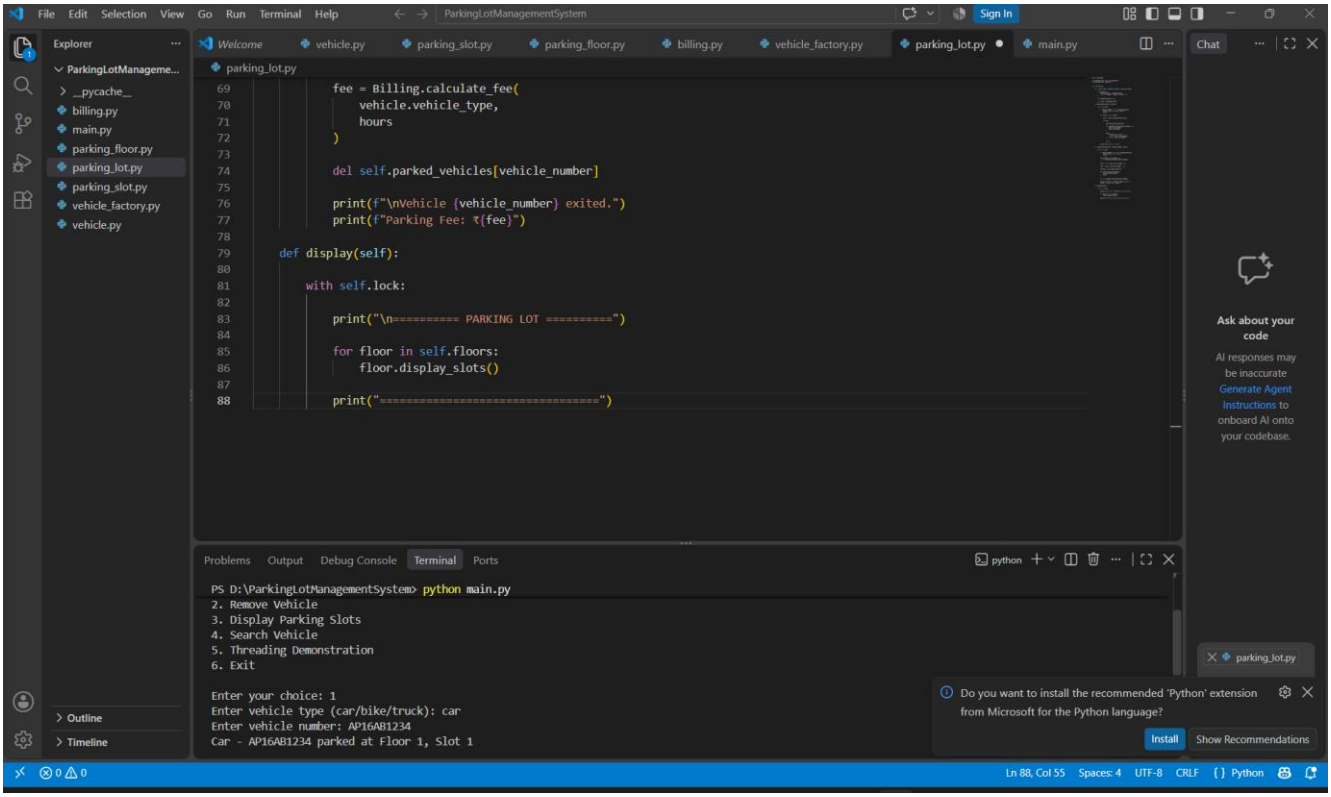


Figure 6.1: Successful Vehicle Entry and Automatic Parking-Slot Allocation

For example, when a car with registration number AP16AB1234 is entered, the system assigns an available slot and displays the floor and slot number. This confirms that the vehicle has been successfully parked and its location has been recorded.

The output demonstrates the successful interaction between the vehicle-management and parking-slot modules and shows how the system reduces manual effort in allocating parking spaces.

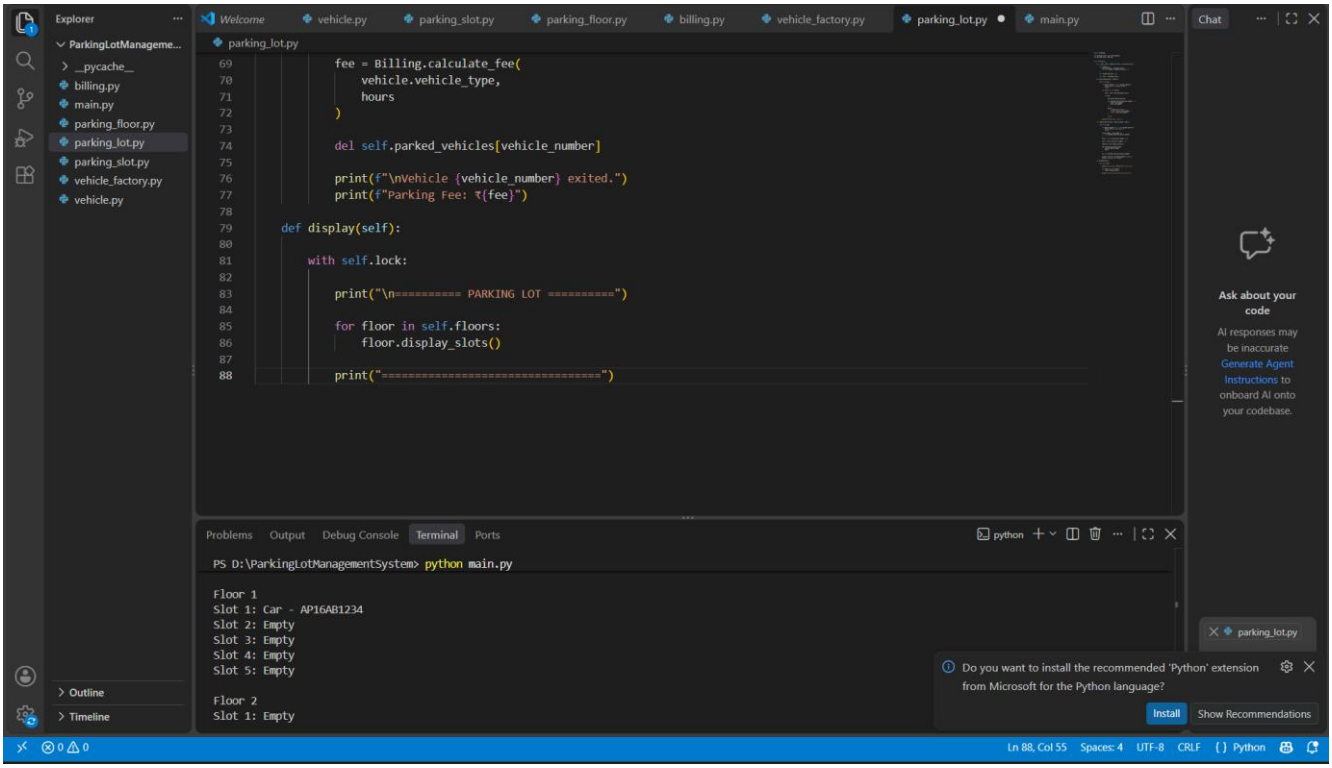


Figure 6.2: Displaying Parking-Slot Availability and Occupancy Status

This output demonstrates the parking-slot status display functionality of the Parking Lot Management System. The user selects option 3 from the main menu to view the available and occupied parking slots. The system displays the slots on each floor and indicates whether each slot is empty or occupied by a vehicle.

After parking the car with registration number AP16AB1234, the assigned slot should be displayed as occupied, while the remaining unused slots should be displayed as available. This functionality helps users understand the current parking capacity and identify empty slots quickly.

The output verifies that the system maintains parking-slot occupancy information and displays the current status of the parking facility. It also demonstrates the role of the slot-management module in organising vehicles across multiple floors.

7 Oop And Design Patterns

7.1 Object-Oriented Programming

Object-Oriented Programming (OOP) organizes a program using classes and objects. It improves modularity, code reuse and maintainability. The Parking Lot Management System demonstrates the following OOP concepts.

Encapsulation

A parking slot maintains its own occupancy information and provides methods for parking or removing a vehicle. This keeps related data and operations together.

Inheritance

Car, Bike and Truck inherit common vehicle characteristics from the base Vehicle class. This reduces duplication and provides a common model for different vehicle categories.

Polymorphism

Different vehicle subclasses can be treated through a common vehicle interface. Where vehicle-specific behaviour or properties differ, the corresponding subclass can provide its own implementation.

Abstraction

The parking-lot module exposes operations such as parking, searching and removing vehicles without requiring the user to manage the internal slot data directly.

7.2 Factory Design Pattern

The Factory Design Pattern centralizes object creation. Instead of creating vehicle objects throughout the application, the program uses a factory to select and create the appropriate vehicle class. For example:

Input: Car

Output: Car object

Input: Bike

Output: Bike object

Input: Truck

Output: Truck object

The factory makes it easier to add additional vehicle categories in the future.

7.3 Separation of Responsibilities

The system uses separate modules for vehicle management, parking, slot management and billing. This separation improves readability and makes testing easier.

7.4 Billing Logic

The current illustrative hourly rates are:

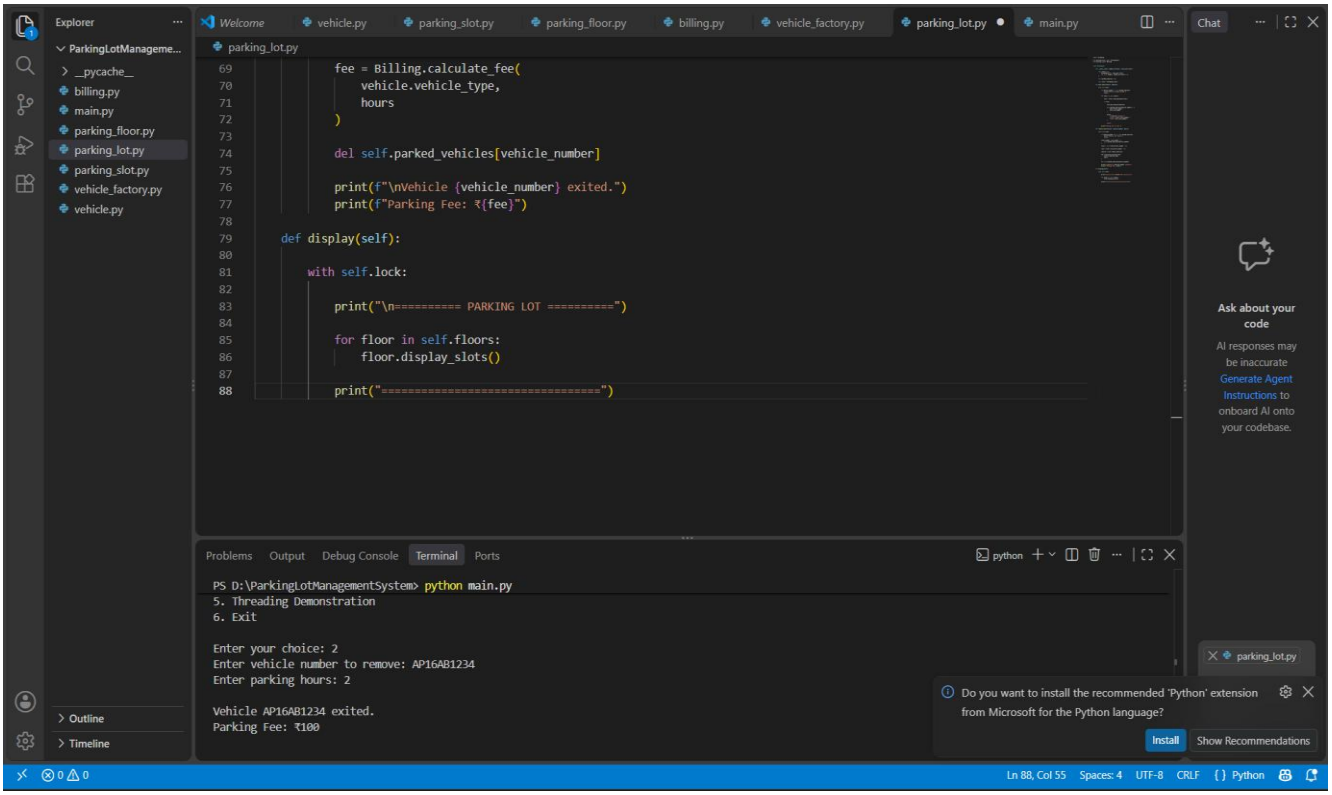
Vehicle Type	Rate per Hour
Bike	₹20
Car	₹50
Truck	₹100

For example, a car parked for two hours incurs a charge of: **Parking Fee = ₹50 × 2 = ₹100** The exact billing calculation and rounding behaviour should match the implementation in billing.py. If the code uses a minimum charge, rounding or a different duration rule, document that actual rule instead.

Figure 7.4.1: Parking Fee Calculation for a Parked Vehicle

This output demonstrates the parking fee calculation functionality of the Parking Lot Management System. When a vehicle is parked, the system identifies its vehicle type and applies the corresponding parking rate to calculate the fee based on the parking duration.

For example, if a car with registration number AP16AB1234 is parked for two hours at an illustrative rate of ₹50 per hour, the total parking fee is ₹100.



8 Concurrency And Thread Synchronization

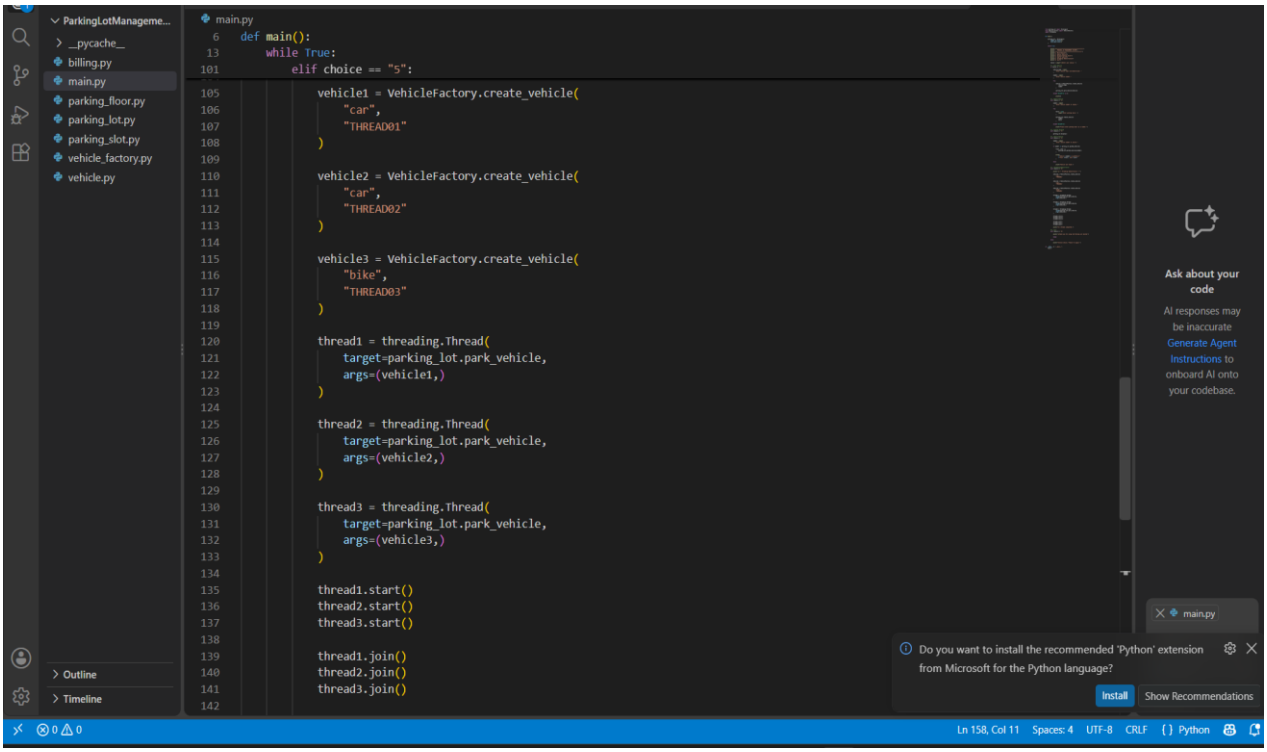
8.1 Introduction to Concurrency

Concurrency allows multiple tasks to make progress during overlapping periods. In a parking facility, multiple vehicles may attempt to enter at approximately the same time. If two operations modify shared parking information without coordination, both might attempt to use the same slot or cause inconsistent records.

8.2 Multithreading in the Project

Python's `threading` module is used to demonstrate concurrent parking operations. The threading demonstration creates separate threads for sample vehicles, starts the threads and waits for them to finish using join ( ).

Illustrative example:



8.3 Thread Synchronization Using Lock

The parking-lot module uses threading. Lock ( ) to protect critical operations that access shared parking data. A lock ensures that only one thread at a time can execute a protected critical section. This helps prevent conflicting updates to parking slots and vehicle-location records.

8.4 Why Synchronization Is Required

Suppose only one parking slot is available and two vehicles attempt to occupy it simultaneously. Without synchronization, both operations could potentially observe the slot as available before either updates the occupancy state. This is a race condition.

9 Testing And Quality Engineering

9.1 Testing Objective

Testing verifies that the system performs the expected operations and handles invalid situations appropriately. The project should be tested for successful parking, vehicle search, removal, billing, parking capacity and concurrent access.

9.2 Test Cases

Test ID	Test Scenario	Expected Result
TC01	Park a car	Car is assigned an available slot
TC02	Park a bike	Bike is assigned an available slot
TC03	Park a truck	Truck is assigned an available slot
TC04	Enter duplicate vehicle number	Duplicate entry rejected
TC05	Park when all slots are full	Full-capacity message displayed
TC06	Search for a parked vehicle	Vehicle location is displayed
TC07	Search for an unknown vehicle	Vehicle-not-found message
TC08	Remove a vehicle	Slot released and fee calculated
TC09	Enter invalid menu input	Invalid input handled
TC10	Run two parking threads	Concurrent execuation handled correctly

9.3 Billing Verification

Using the illustrative rates:

Vehicle Type	Duration	Expected Fee
Bike	2 hours	₹40
Car	2 hours	₹100
Truck	2 hours	₹200

These expected values apply when the program calculates the fee as hourly rate multiplied by the entered duration, without additional minimum charges or rounding rules.

9.4 Quality Engineering

Quality engineering involves more than checking whether the program runs. It includes validating inputs, checking boundary conditions, testing module behaviour and reviewing whether errors are handled safely. Recommended checks include:

9.4.1 Empty or malformed registration numbers.

9.4.2 Unsupported vehicle types.

9.4.3 Negative or non-integer parking durations.

9.4.4 Attempts to remove a vehicle that is not parked.

9.4.5 Repeated parking and removal operations.

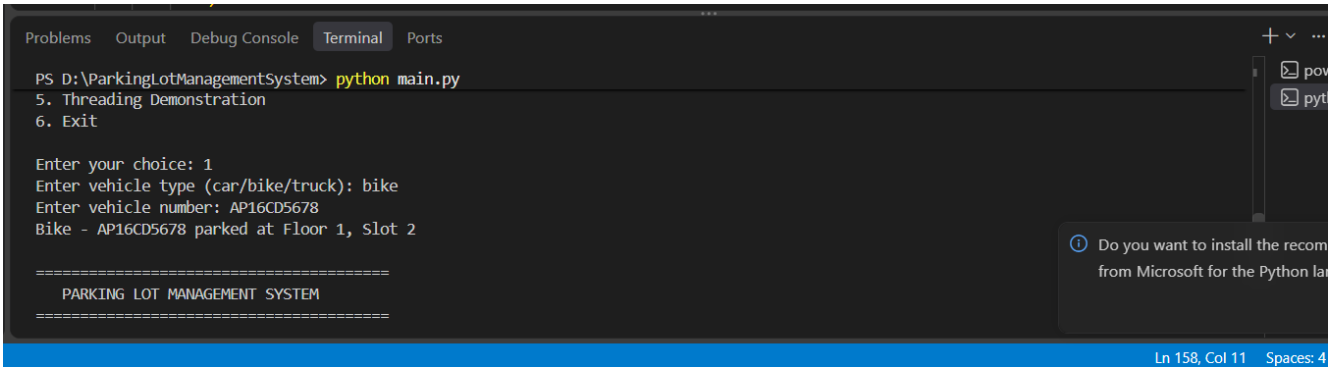
9.4.6 Simultaneous parking attempts.

9.5 Test Results

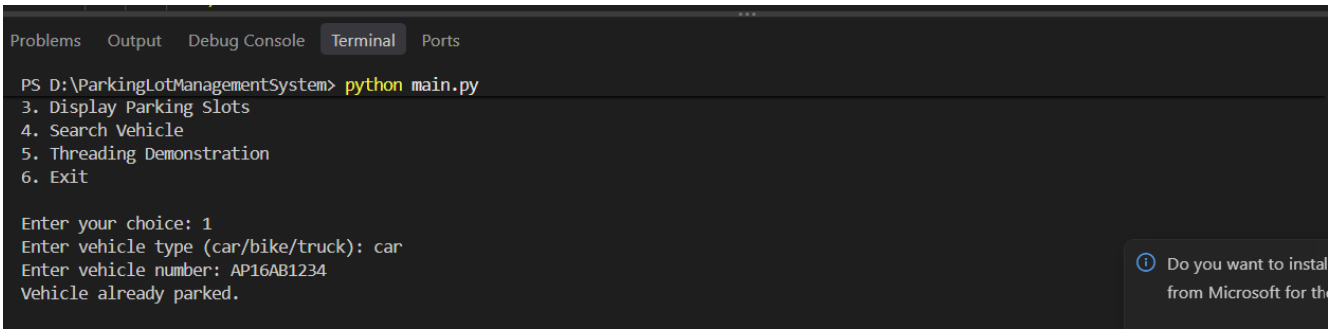
Record the actual result after executing every test case.

Test ID	Actual Result	Status
TC01	Car parked successfully	Pass
TC02	Bike parked successfully	Pass
TC03	Truck parked successfully	Pass
TC04	Duplicate vehicle entry validation	Pass
TC05	Parking slot availability	Pass
TC06	Vehicle removal and fee calculation	Pass
TC07	Exit	Pass

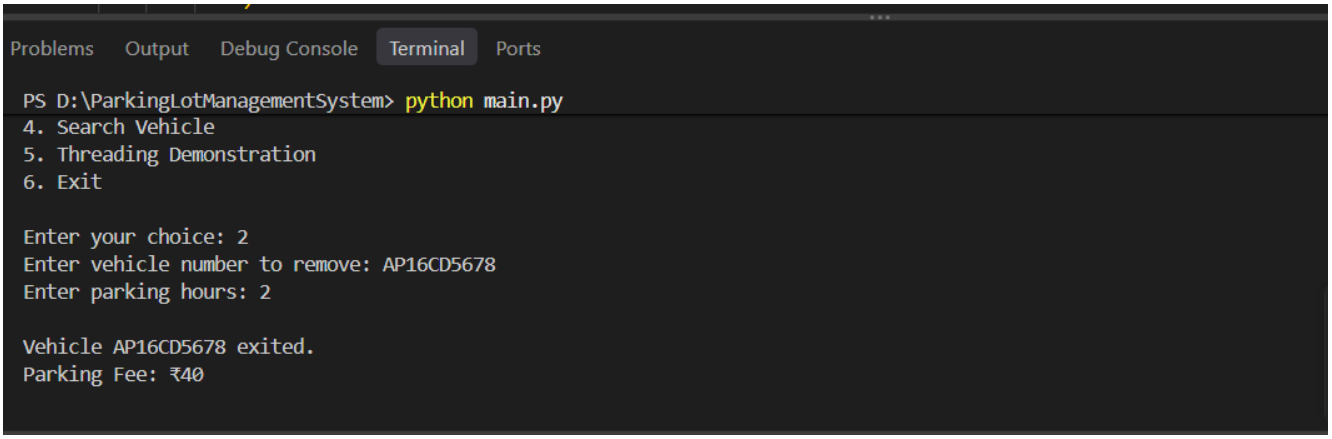
TC 02 – Bike parked successfully



TC 04 – Duplicate vehicle entry validation



TC 06 – Vehical removal and fee calculation



10 Ci/Cd And Containerisation

10.1 Introduction

Continuous Integration and Continuous Delivery or Deployment (CI/CD) are software engineering practices that automate testing and delivery. Containerisation packages an application and its dependencies into a portable environment. These practices can improve consistency when a project is developed by multiple people or deployed across different systems.

10.2 Applicability to the Project

The current Parking Lot Management System is a Python console application. It can be tested locally without a CI/CD pipeline or Docker. However, the project can be extended by adding automated tests, a GitHub Actions workflow and a Docker image.

10.3 Proposed CI Workflow

A possible workflow is:

1. Developer updates the Python source code.
2. Changes are committed and pushed to GitHub.
3. GitHub Actions starts a workflow.
4. The workflow installs the required Python version.
5. Automated tests execute.
6. The workflow reports whether the tests passed.

11 Ai-Assisted Development And Code Review

11.1 Introduction

AI-assisted development tools can support programming by explaining code, suggesting improvements, identifying potential defects and helping developers create test cases. In this project, AI assistance can be used to understand Python classes, review exception handling, explain thread synchronization and identify missing test scenarios.

11.2 Code Explanation

AI tools can explain how the vehicle factory, parking-slot allocation and billing modules interact.

This is especially useful for understanding a modular Python application.

Error Analysis

When a runtime error occurs, the developer can provide the error message and relevant source code to obtain possible explanations and debugging suggestions.

Test-Case Generation

AI tools can suggest scenarios such as duplicate vehicle registration, parking-full conditions, invalid vehicle types and simultaneous entry operations.

Code Review

AI-assisted review can highlight possible issues involving shared state, incorrect input handling, duplicate records or inconsistent slot updates.

11.3 Evidence of AI-Assisted Development

If AI assistance was used, include a short description of the task, the suggestion received, the changes accepted or rejected, and the tests used to verify the changes. If no AI review was performed, present this section as a proposed development.

12 Performance Evaluation And Live Demonstration

12.1 Performance Evaluation

Performance evaluation studies how the application behaves as the number of parking floors, slots and vehicle operations increases. The current implementation uses an ordered search for available slots and a dictionary for parked-vehicle location records. Theoretical complexity provides an estimate of how the work grows with input size. Actual performance should be measured by running the program under controlled conditions.

12.2 Evaluation Metrics

Metric	Measurement Method
Slot allocation time	Measure the time required to park a vehicle and assign a slot
Vehicle lookup time	Measure the time required to search a registration number
Vehicle removal time	Measure the time for remove a vehicle and billing
Capacity handling	Test with all slots are occupied
Concurrent operations	Observe completion of thread-based parking attempts
Correctness	Compare actual results with expected results

12.3 Performance Results Table

Complete the following table using actual measurements.

Experiment	Test condition	Expected Result	Actual Result
Vehicle Slot allocation	Car – AP16AB1234	Vehicle assigned to the first available slot	Allocation time: 0.341 ms, Successfully parked at Floor 1
Vehicle Search	Search for vehicle AP16AB1234	Display its floor number and parking slot number	AP16AB1234 was successfully found at Floor1, Slot1
Vehicle Removal	Remove AP16AB1234 after entering parking hours	Vehicle should be removed and parking fee calculated correctly	Vehicle AP16AB1234 exited successfully. Parking fee: ₹100.
Parking Slot Capacity	Display all parking slots	System should display occupied and empty slots correctly	Floor 1 displayed three occupied slots and two empty slots. Floor 2 displayed five empty slots.

Threading Demonstration	Excute three concurrent vehicle parking requests	Threads should complete and vehicles should be assigned available slots.	Three vehicles (THREAD01, THREAD02, THREAD03) were successfully parked in Floor 1, Slots 1, 2, and 3. All threads completed successfully.

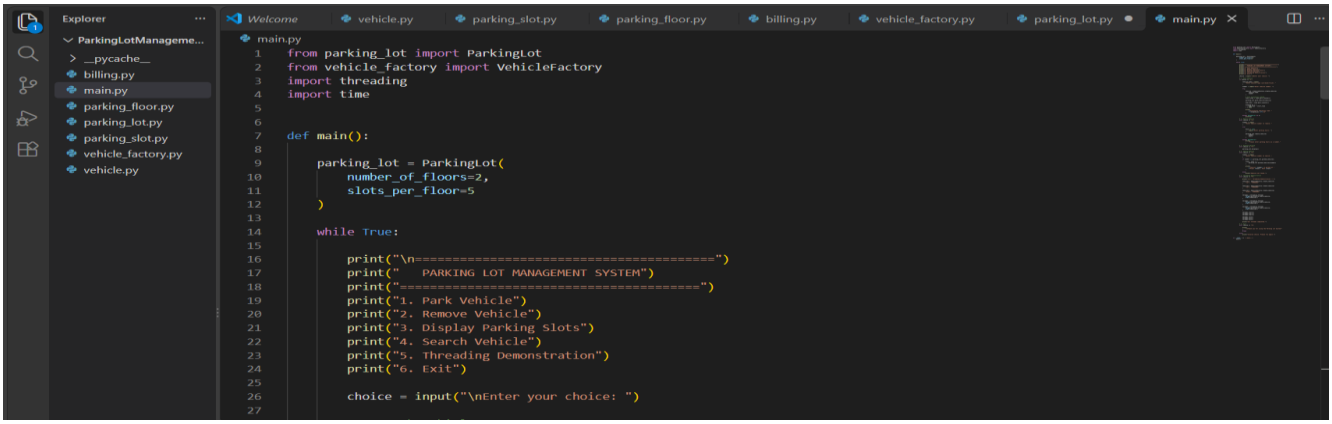
12.4 Live Demonstration Procedure

The following sequence demonstrates the complete project workflow.

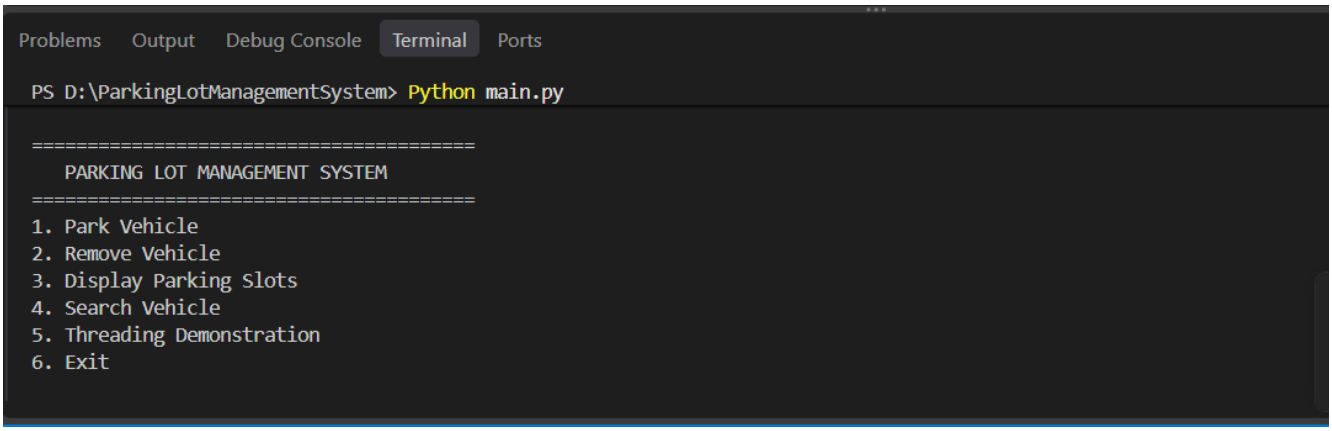
1. Open the project in Visual Studio Code.
2. Run **python main.py**
3. Select **Park Vehicle** and enter a car registration number.
4. Park a bike and a truck.
5. Select **Display Parking Slots** and show occupied slots.
6. Select **Search Vehicle** and enter a registered number.
7. Select **Remove Vehicle**, enter the exact registration number and provide the parking duration.
8. Show the calculated fee and released slot.
9. Run the threading demonstration.
10. Discuss the test cases and the limitations of the current implementation.

13. Recommend Screenshots

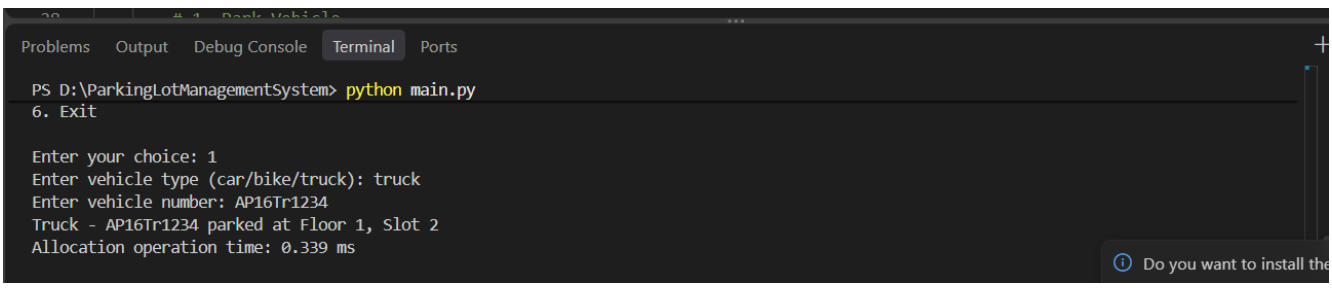
• Screenshot 1: Project folder and Python files.

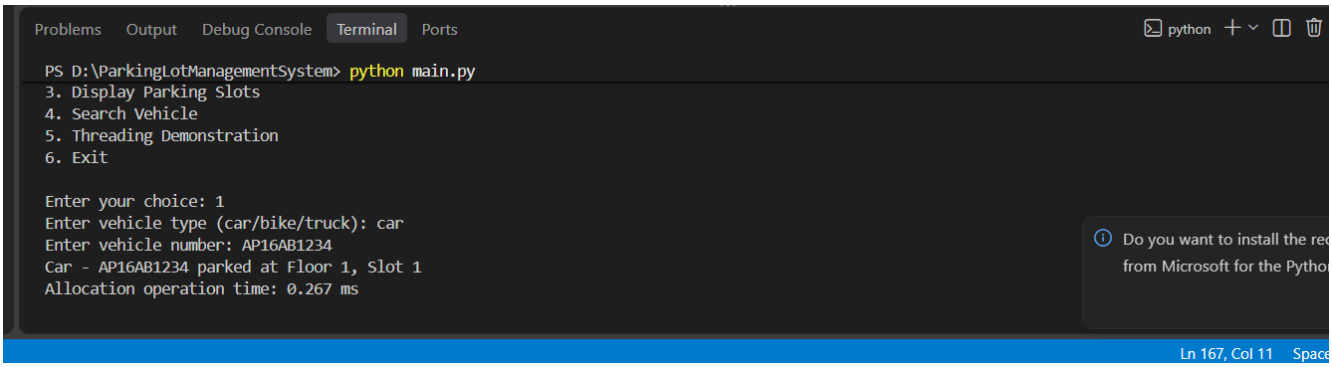


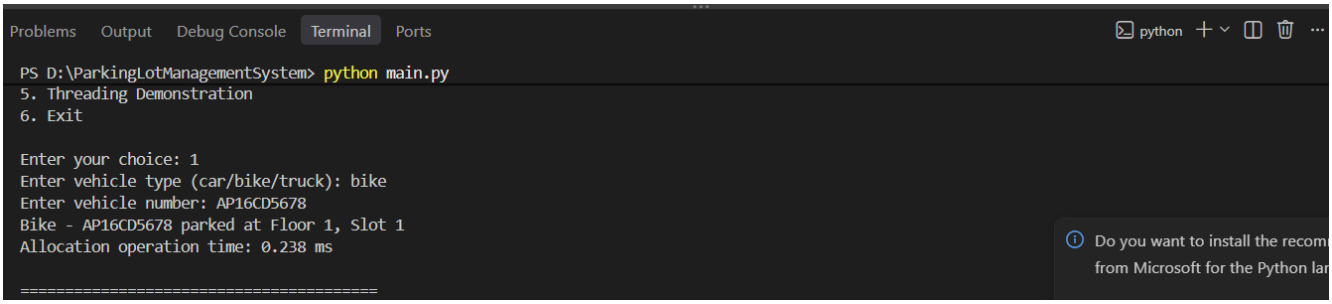
• Screenshot 2: Application menu.



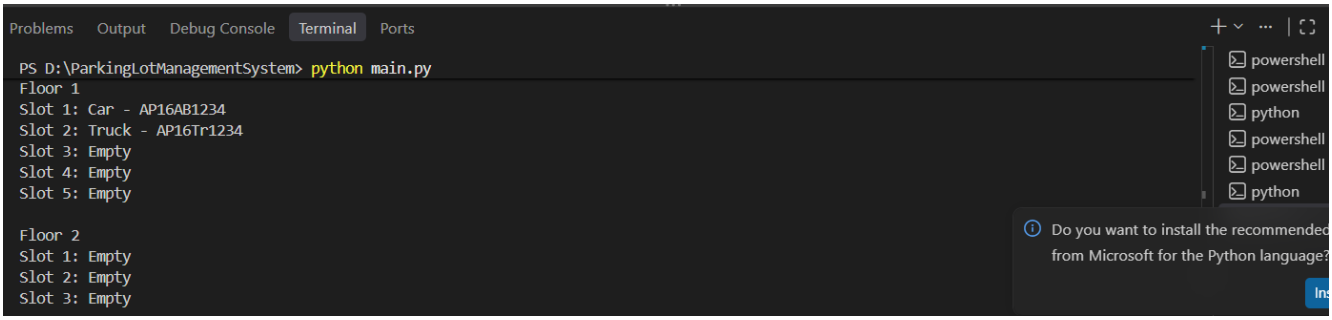
• Screenshot 3: Successful vehicle parking.



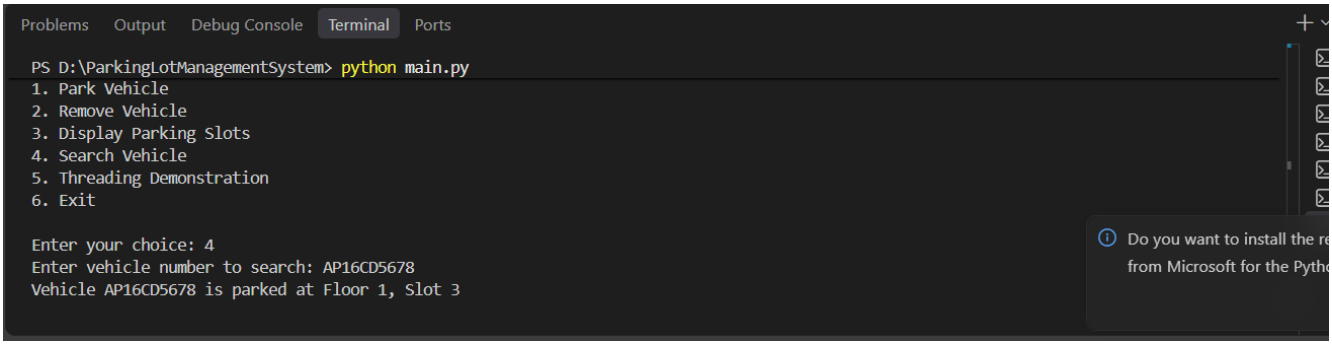


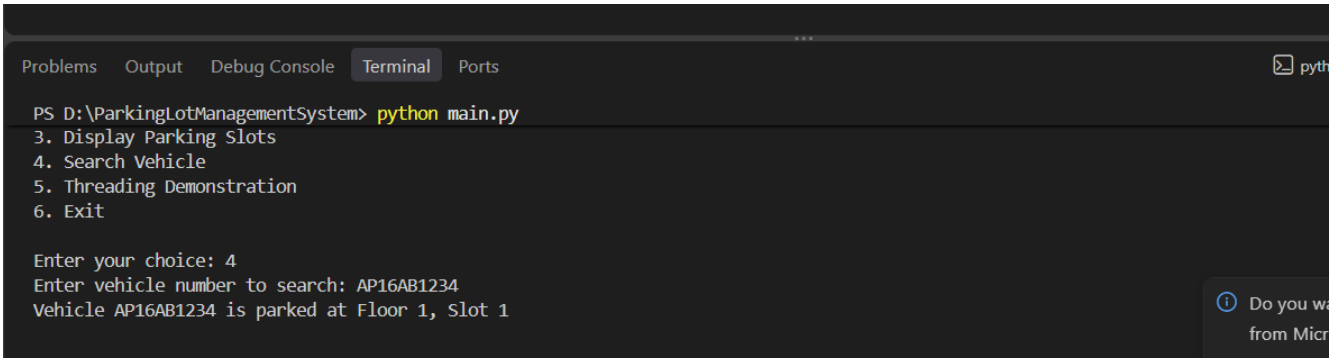


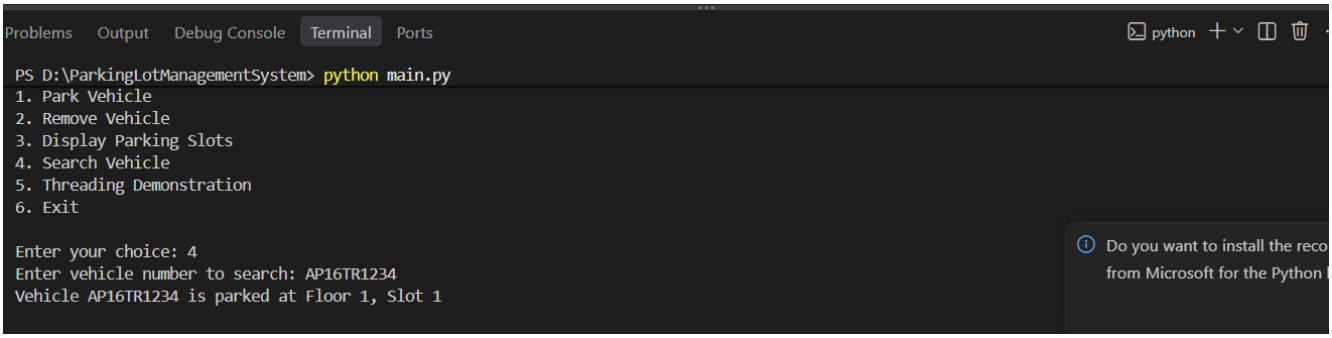
• Screenshot 4: Parking-slot display.



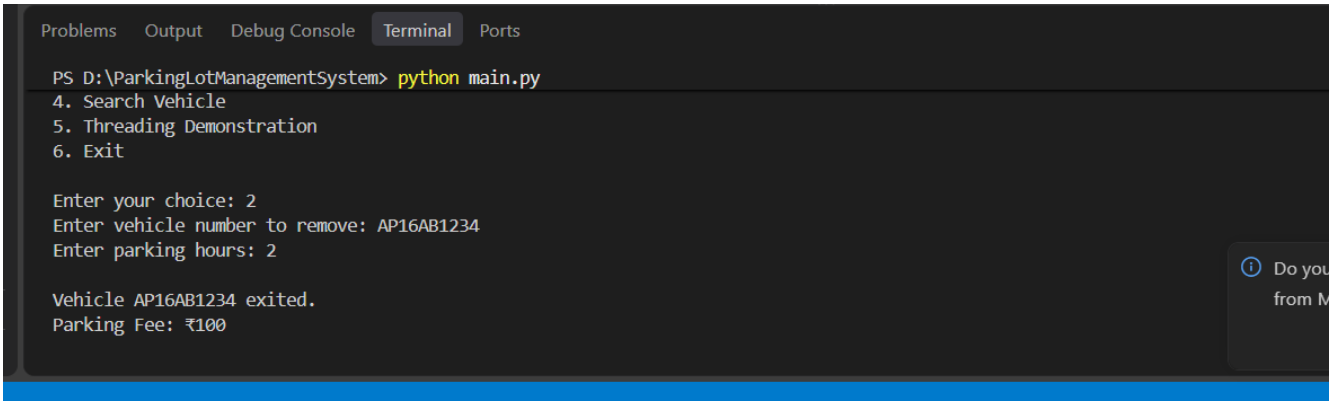
• Screenshot 5: Vehicle search result.



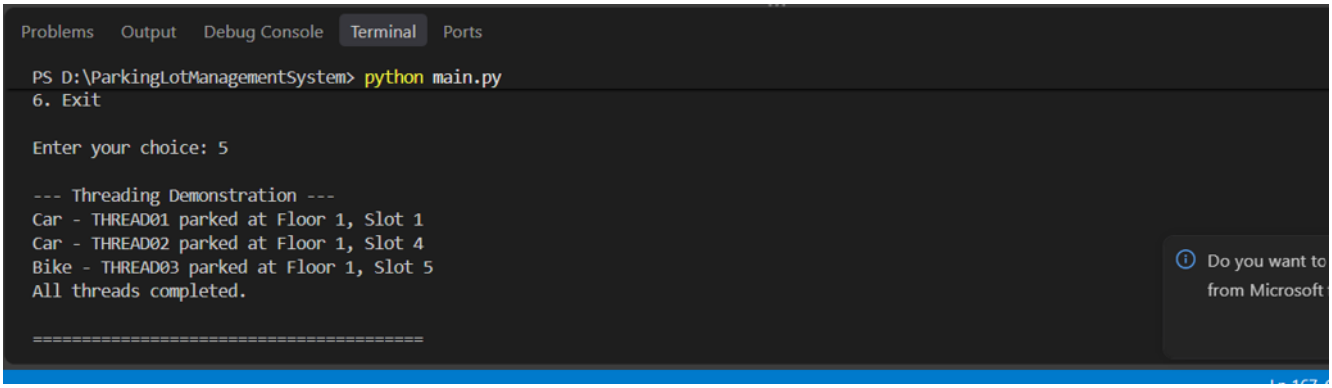




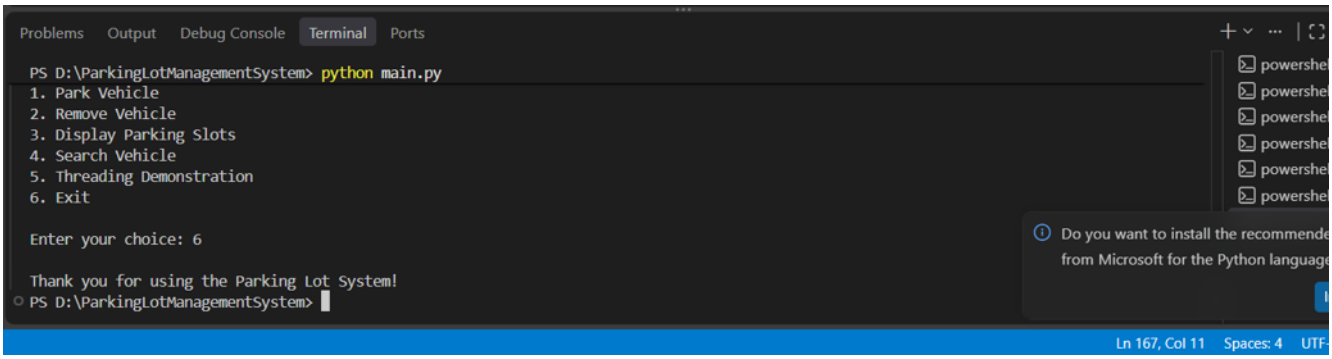
• Screenshot 6: Vehicle removal and billing.



Screenshot 7: Threading demonstration



Screenshot 8: Exit



1. **Screenshot 1:** Displays the project folder containing the Python source files.
2. **Screenshot 2:** Displays the main menu with all available application operations.
3. **Screenshot 3:** Shows successful vehicle parking with the assigned floor and slot number.
4. **Screenshot 4:** Displays the occupied and available parking slots.
5. **Screenshot 5:** Shows the search result for the requested vehicle.
6. **Screenshot 6:** Shows successful vehicle removal and the calculated parking fee.
7. **Screenshot 7:** Demonstrates multiple vehicle parking operations using threading.
8. **Screenshot 8:** Shows the application exiting successfully.

14 Conclusion, Future Scope And References

14.1 Conclusion

The Parking Lot Management System demonstrates how a practical parking-management problem can be addressed through a modular Python application. The system organises parking slots into multiple floors, represents vehicle categories using object-oriented programming and allocates the first available slot through a greedy search strategy. It maintains registration numbers and vehicle locations using a dictionary and calculates parking charges according to the vehicle category and parking duration. The Factory Design Pattern separates vehicle creation from the main parking logic. Multithreading demonstrates how parking operations can be initiated concurrently, while a lock protects critical shared operations. The project also provides a foundation for studying software testing, algorithmic complexity, code review and performance evaluation. The current

console-based implementation fulfils the main educational objectives of demonstrating algorithms, data structures, modular programming, OOP and basic concurrency. Additional work is required before the application can be considered a production-ready parking-management platform.

14.2 Limitations

- The application uses a console-based interface.
- Parking records are maintained in memory and may not persist after the program closes.
- Billing uses the duration supplied by the user rather than an automatically recorded entry and exit time.
- The greedy strategy does not optimise assignments using vehicle dimensions, reserved spaces or walking distance.
- The application does not include a payment gateway, hardware sensors or a web-based dashboard.
- The threading demonstration does not establish scalability under production traffic.

14.3 Future Scope

1. Integrate a database to preserve parking records.
2. Develop a graphical or web-based interface.
3. Record entry and exit timestamps automatically.
4. Integrate QR-code scanning or number-plate recognition.
5. Add payment gateway integration.
6. Display live parking availability.

7. Add reserved slots and vehicle-size constraints.
8. Implement automated unit and integration tests.
9. Configure CI/CD using GitHub Actions.
10. Package and test the application using Docker.
11. Conduct systematic performance and concurrency testing.

14.4 Technical Defence

During the project viva, the developer should be able to explain:

- Why a greedy algorithm is used for slot allocation.
- Why a dictionary is suitable for registration-number lookup.
- How inheritance represents cars, bikes and trucks.
- Why a factory is used to create vehicle objects.
- What a race condition is and how a lock helps prevent conflicting updates.
- How the billing module calculates parking charges.
- What the system does when all slots are occupied.
- How the test cases verify expected behaviour.
- What the time complexity of slot allocation is.
- Which features are implemented and which remain future work.

15.5 References

1. Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/

2. Python Software Foundation, “Threading — Thread-Based Parallelism.” [Online]. Available: https://docs.python.org/3/library/threading.html

3. Python Software Foundation, “Data Structures.” [Online]. Available: https://docs.python.org/3/tutorial/datastructures.html

4. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.